

## 과제4 보고서, 김도훈, 2026406041

### <목차>

1. 3-1 설명, 구현, 실행 결과
2. 3-2 설명, 구현, 실행 결과

### 1. 3-1

#### 3-1-1 설명

과제 3-1은 3차원 공간에서 움직이는 2축 매니퓰레이터이다. 이 과제는 이 매니퓰레이터 FK, IK를 유도하여 실제로 계산하여 값들을 비교해보는 과제이다.

#### 3-1-2 구현

##### 사용한 식

##### FK

$$x = L_2 \cos \theta_1 \cos \theta_2, \quad y = L_2 \sin \theta_1 \cos \theta_2, \quad z = L_1 + L_2 \sin \theta_2$$

##### IK

##### 해석

$$\theta_1 = \text{atan2}(y, x), \quad \theta_2 = \text{atan2}(z - L_1, \sqrt{x^2 + y^2})$$

##### 수치

자코비안은 FK식을 각각  $\theta_1, \theta_2$ 에 대해 편미분하여 구하였다.

$$J = \begin{bmatrix} -L_2 \sin \theta_1 \cos \theta_2 & -L_2 \cos \theta_1 \sin \theta_2 \\ L_2 \cos \theta_1 \cos \theta_2 & -L_2 \sin \theta_1 \sin \theta_2 \\ 0 & L_2 \cos \theta_2 \end{bmatrix}$$

링크 값, 초기 각도 값들을 변수로 지정 후 FK를 통해 end-effector의 위치를 구한다. 그리고 구한 위치를 바탕으로 IK 계산을 하여 각도 값들을 구하도록 구현했다.

```
clc;
clear;
close all;

%% 링크 길이
L1 = 5;
L2 = 20;

%% FK에 사용할 관절각
theta1 = deg2rad(30);
theta2 = deg2rad(40);

% FK
Px = L2*cos(theta1)*cos(theta2);
Py = L2*sin(theta1)*cos(theta2);
Pz = L1 + L2*sin(theta2);

fprintf('Problem 3-1\n');

fprintf('\n[Original Joint Angle]\n');
fprintf('theta1 = %.4f deg\n', rad2deg(theta1));
fprintf('theta2 = %.4f deg\n', rad2deg(theta2));

fprintf('\n[Forward Kinematics]\n');
fprintf('Px = %.4f\n', Px);
fprintf('Py = %.4f\n', Py);
fprintf('Pz = %.4f\n', Pz);
```

```
% IK 수치

% 초기 관절각
q = deg2rad([10; 10]);
% 목표 End-Effector 위치
Pd = [Px; Py; Pz];
% 최대 반복 횟수
max_iter = 1000;
% 허용 오차
tol = 1e-8;
% Step size
alpha = 0.5;

for i = 1:max_iter
    th1 = q(1);
    th2 = q(2);
    % 현재 관절각에서 FK 계산
    Px_now = L2*cos(th1)*cos(th2);
    Py_now = L2*sin(th1)*cos(th2);
    Pz_now = L1 + L2*sin(th2);
    P_now = [Px_now; Py_now; Pz_now];
    % 목표 위치와 현재 위치의 오차
    error = Pd - P_now;
    % 오차가 충분히 작으면 종료
    if norm(error) < tol
        break;
    end
    % Jacobian
    J = [ ...
        -L2*sin(th1)*cos(th2), -L2*cos(th1)*sin(th2);
        L2*cos(th1)*cos(th2), -L2*sin(th1)*sin(th2);
        0, L2*cos(th2)
    ];
    % Jacobian 역행렬
    dq = pinv(J) * error;
    % 관절각 갱신
    q = q + alpha*dq;
end
```

```
% IK 해석

% theta1 계산
theta1_alg = atan2(Py, Px);

% XY 평면에서의 거리
r = sqrt(Px^2 + Py^2);

% theta2 계산
theta2_alg = atan2(Pz - L1, r);

fprintf('\n[Algebraic IK]\n');
fprintf('theta1 = %.4f deg\n', rad2deg(theta1_alg));
fprintf('theta2 = %.4f deg\n', rad2deg(theta2_alg));
```

코드에서도 알 수 있듯이 링크1, 링크2 길이는 각각 5, 20으로 설정했고 초기 FK 계산에 사용될 각도는 30도, 40도로 설정했다.

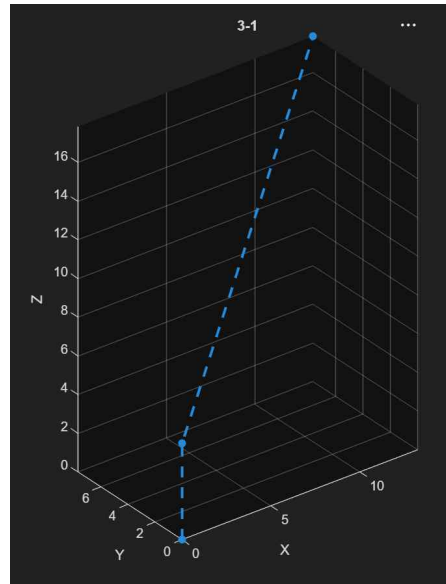
### 3-1-3 실행 결과

```
Problem 3-1

[Original Joint Angle]
theta1 = 30.0000 deg
theta2 = 40.0000 deg

[Forward Kinematics]
Px = 13.2683
Py = 7.6604
Pz = 17.8558
|
[Algebraic IK]
theta1 = 30.0000 deg
theta2 = 40.0000 deg

[Numerical IK]
theta1 = 30.0000 deg
theta2 = 40.0000 deg
Iteration = 32
Final Error = 0.000000064
>>
```



초기 각도가 30도 40도 설정했다. 수치, 해석 IK 계산 결과가 모두 기존의 각도와 똑같은 것을 확인할 수 있다.

## 2. 3-2

### 3-2-1 설명

과제 3-2는 3개의 관절을 가지고 있는 2차원 매니퓰레이터이다. 3-1과 주어진 매니퓰레이터만 달라지고 똑같은 과제이다.

### 3-2-2 구현

#### 사용한식

#### FK

$$x = L_1 \cos \theta_1 + L_2 \cos(\theta_1 + \theta_2) + L_3 \cos(\theta_1 + \theta_2 + \theta_3), y = L_1 \sin \theta_1 + L_2 \sin(\theta_1 + \theta_2) + L_3 \sin(\theta_1 + \theta_2 + \theta_3)$$

end-effector의 방향 =  $\theta_1 + \theta_2 + \theta_3 = \phi$

#### IK

#### 해석

$$x_3 = x_{end} - L_3 \cos \phi, y_3 = y_{end} - L_3 \sin \phi \rightarrow \text{joint3 위치 계산(end-effector 위치를 사용해서)}$$

$$\cos \theta_2 = \frac{x_3^2 + y_3^2 - L_1^2 - L_2^2}{2L_1 L_2}, \sin \theta_2 = +\sqrt{1 - \cos^2 \theta_2} \rightarrow \theta_2 = \text{atan2}(\sin \theta_2, \cos \theta_2)$$

$$\theta_1 = \text{atan2}(y_3, x_3) - \text{atan2}(L_2 \sin \theta_2, L_2 \cos \theta_2), \quad \theta_3 = \phi - \theta_1 - \theta_2$$

## 수치

### 자코비안 행렬

$$J = \begin{bmatrix} -L_1 \sin \theta_1 - L_2 \sin(\theta_1 + \theta_2) - L_3 \sin(\theta_1 + \theta_2 + \theta_3) & -L_2 \sin(\theta_1 + \theta_2) - L_3 \sin(\theta_1 + \theta_2 + \theta_3) & -L_3 \sin(\theta_1 + \theta_2 + \theta_3) \\ L_1 \cos \theta_1 + L_2 \cos(\theta_1 + \theta_2) + L_3 \cos(\theta_1 + \theta_2 + \theta_3) & L_2 \cos(\theta_1 + \theta_2) + L_3 \cos(\theta_1 + \theta_2 + \theta_3) & L_3 \cos(\theta_1 + \theta_2 + \theta_3) \\ 1 & 1 & 1 \end{bmatrix}$$

```
clc;
clear;
close all;

% Problem 3-2

% Link Length
l1 = 15;
l2 = 20;
l3 = 10;

% Joint Angle
theta1 = deg2rad(30);
theta2 = deg2rad(45);
theta3 = deg2rad(-15);

% FK
Px = l1*cos(theta1) ...
    + l2*cos(theta1 + theta2) ...
    + l3*cos(theta1 + theta2 + theta3);

Py = l1*sin(theta1) ...
    + l2*sin(theta1 + theta2) ...
    + l3*sin(theta1 + theta2 + theta3);

% End-Effector 방향
phi = theta1 + theta2 + theta3;

%% 결과 출력
fprintf('Problem 3-2\n');
fprintf('Original Joint Angle\n');
fprintf('theta1 = %.4f deg\n', rad2deg(theta1));
fprintf('theta2 = %.4f deg\n', rad2deg(theta2));
fprintf('theta3 = %.4f deg\n', rad2deg(theta3));
fprintf('Forward Kinematics\n');
fprintf('Px = %.4f\n', Px);
fprintf('Py = %.4f\n', Py);
fprintf('phi = %.4f deg\n', rad2deg(phi));
```

```
% IK 해석
% Joint 3 위치 계산
Px_joint3 = Px - l3*cos(phi);
Py_joint3 = Py - l3*sin(phi);

fprintf('\n[Joint 3 Position]\n');
fprintf('Px_joint3 = %.4f\n', Px_joint3);
fprintf('Py_joint3 = %.4f\n', Py_joint3);

% theta2 계산
c2 = (Px_joint3^2 + Py_joint3^2 ...
    - l1^2 - l2^2) ...
    / (2*l1*l2);

% c2가 아주 조금 1보다 커지는 경우 방지 ( MATLAB 수치 계산으로 인해)
c2 = max(-1, min(1, c2));

% 여러 해 중 + 방향 선택
s2 = sqrt(1 - c2^2);
theta2_alg = atan2(s2, c2);

% theta1 계산
theta1_alg = atan2(Py_joint3, Px_joint3) ...
    - atan2(l2*sin(theta2_alg), ...
    l1 + l2*cos(theta2_alg));

% theta3 계산
theta3_alg = phi ...
    - theta1_alg ...
    - theta2_alg;

fprintf('\n[Algebraic IK]\n');

fprintf('theta1 = %.4f deg\n', rad2deg(theta1_alg));
fprintf('theta2 = %.4f deg\n', rad2deg(theta2_alg));
fprintf('theta3 = %.4f deg\n', rad2deg(theta3_alg));
```

```
% IK 수치
% 초기 관절각
q = deg2rad([10;
    20;
    0]);

% 목표 End-Effector 상태
Xd = [Px;
    Py;
    phi];

% 최대 반복 횟수
max_iter = 1000;

% 허용 오차
tol = 1e-8;

% Step Size
alpha = 0.5;

for i = 1:max_iter

    %% 현재 관절각

    th1 = q(1);
    th2 = q(2);
    th3 = q(3);

    % 현재 관절각에서 FK 계산
    Px_now = l1*cos(th1) ...
        + l2*cos(th1 + th2) ...
        + l3*cos(th1 + th2 + th3);

    Py_now = l1*sin(th1) ...
        + l2*sin(th1 + th2) ...
        + l3*sin(th1 + th2 + th3);
```

```
% Jacobian
J = [ ...
    -l1*sin(th1)-l2*sin(th1+th2)-l3*sin(th1+th2+th3),
    -l2*sin(th1+th2)-l3*sin(th1+th2+th3), ...
    -l3*sin(th1+th2+th3); ...

    l1*cos(th1)+l2*cos(th1+th2)+l3*cos(th1+th2+th3),
    l2*cos(th1+th2)+l3*cos(th1+th2+th3), ...
    l3*cos(th1+th2+th3); ...

    1, 1, 1
];

% 관절각 변화
dq = pinv(J) * error;
% 관절각 Update
q = q + alpha*dq;

end

%% Numerical IK 결과

fprintf('\n[Numerical IK]\n');

fprintf('theta1 = %.4f deg\n', rad2deg(q(1)));
fprintf('theta2 = %.4f deg\n', rad2deg(q(2)));
fprintf('theta3 = %.4f deg\n', rad2deg(q(3)));

fprintf('Iteration = %d\n', i);
fprintf('Final Error = %.10f\n', norm(error));
```

### 3-2-2 실행 결과

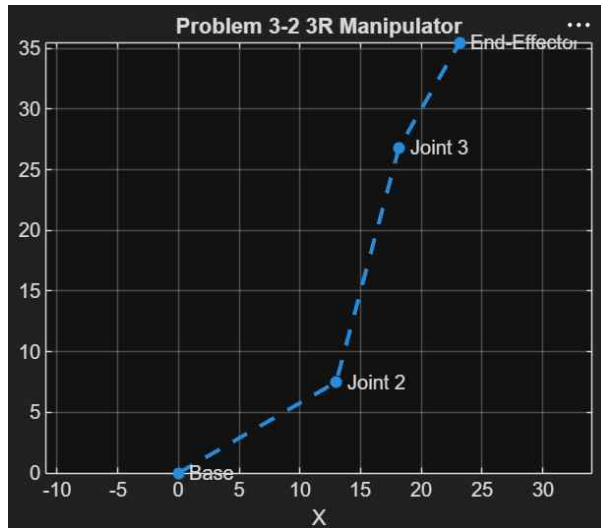
```
[Original Joint Angle]
theta1 = 30.0000 deg
theta2 = 45.0000 deg
theta3 = -15.0000 deg

[Forward Kinematics]
Px = 23.1668
Py = 35.4788
phi = 60.0000 deg

[Joint 3 Position]
Px_joint3 = 18.1668
Py_joint3 = 26.8185

[Algebraic IK]
theta1 = 30.0000 deg
theta2 = 45.0000 deg
theta3 = -15.0000 deg

[Numerical IK]
theta1 = 30.0000 deg
theta2 = 45.0000 deg
theta3 = -15.0000 deg
Iteration = 33
Final Error = 0.0000000085
```



FK 계산을 위해 변수로 설정한 30도 45도 -15도가 IK 계산시에도 잘 나오는 걸 확인할 수 있다.